AI Poker Arena — Spec Sheet v1.0

> Autonomous AI agents play Texas Hold'em poker against each other, with user-defined strategies and live trash talk.

---

1. Overview

A Python CLI application that runs a full game of No-Limit Texas Hold'em between 3-4 AI agents. Each agent is powered by an LLM and follows a user-defined strategy written in a Markdown file. Agents make betting decisions AND communicate with each other through a live chat feed — bluffing, trash talking, and lying about their hands in real time.

Core loop: Deal → Agents think → Agents chat → Agents act → Repeat until winner.

---

2. Architecture

```
poker-agents/
├── SPEC.md # This file
├── main.py # Entry point — runs the game
├── engine/
│ ├── __init__.py
│ ├── game.py # Game loop, round management
│ ├── deck.py # Card deck, shuffling, dealing
│ ├── hand_evaluator.py # Hand ranking & winner detection
│ ├── pot.py # Pot management, side pots
│ └── types.py # Enums, dataclasses (Card, Hand, Action, etc.)
├── agents/
│ ├── __init__.py
│ ├── base.py # BaseAgent class — LLM integration
│ └── strategies/ # User-editable strategy files
│ ├── shark.md # Aggressive, reads opponents
│ ├── bluffer.md # Chaotic, lies constantly
│ ├── tight.md # Conservative, only plays premiums
│ └── maniac.md # Unpredictable, loves chaos
├── display/
│ ├── __init__.py
│ └── renderer.py # Terminal UI — cards, table, chat feed
├── config.py # Game settings (buy-in, blinds, etc.)
├── requirements.txt # Dependencies
└── README.md # How to run, how to create agents
```

3. Game Rules — No-Limit Texas Hold'em

3.1 Setup

- **Players:** 3-4 AI agents (configurable)
- **Starting chips:** 1,000 each (configurable)
- **Blinds:** Small blind 10, Big blind 20 (configurable)
- **Blind escalation:** None for v1 (cash game style)

3.2 Round Flow

- 1. **Dealer rotation** — Button moves clockwise each hand
- 2. **Blinds posted** — SB and BB forced bets
- 3. **Deal hole cards** — 2 cards to each agent (private)
- 4. **Pre-flop** — Betting round starting left of BB
- 5. **Flop** — 3 community cards revealed → betting round
- 6. **Turn** — 1 community card revealed → betting round
- 7. **River** — 1 community card revealed → betting round
- 8. **Showdown** — Best 5-card hand wins the pot
- 9. **Chat phase** — Agents react to the outcome

3.3 Available Actions

- `fold` — Surrender hand
- `check` — Pass (if no bet to match)
- `call` — Match current bet
- `raise <amount>` — Increase the bet (min raise = previous raise size, max = all-in)
- `all_in` — Bet all remaining chips

3.4 Hand Rankings (standard)

- 1. Royal Flush
- 2. Straight Flush
- 3. Four of a Kind
- 4. Full House
- 5. Flush
- 6. Straight
- 7. Three of a Kind
- 8. Two Pair
- 9. One Pair
- 10. High Card

3.5 Win Condition

- Game is a **single hand** — one deal, one showdown, one winner
- Winner = best 5-card hand at showdown (or last agent standing if all others fold)
- No multi-hand sessions in v1 — keeps it fast and cheap on API costs
- Future: optional multi-hand/tournament mode

---

4. Agent System

4.1 Strategy Files

Each agent's personality and strategy is defined in a Markdown file in agents/strategies/. Users can edit these freely.

Example — shark.md:

```markdown
# Shark 🦈
```

Personality

You are a cold, calculating poker player. You rarely speak unless it's to intimidate. You respect strong players and dismiss weak ones.

Strategy

- Play tight-aggressive: only enter pots with strong hands, but bet big when you do
- Position matters: play looser in late position, tighter in early position
- Pay attention to opponent betting patterns — if someone always bets big, they might be a bluffer
- Semi-bluff with draws (flush draws, straight draws) but never pure bluff with nothing
- Slow-play monster hands occasionally to trap aggressive opponents

Bluffing Style

- Rarely bluff in chat — when you do speak, it's intimidating and short
- If you have a monster hand, stay quiet or say something dismissive
- Never reveal your actual hand in chat
- Example chat: "You sure about that raise?", "...", "Interesting."

Bet Sizing

- Standard raise: 2.5-3x the big blind pre-flop
- Continuation bet: 50-65% of pot on the flop
- Value bet: 60-80% of pot on turn/river with strong hands
- Bluff bet: 33-50% of pot (keep it cheap)

`

Example — **bluffer.md** : ` markdown

The Bluffer 🎭

Personality

You are loud, chaotic, and LOVE lying. You talk constantly. You claim to have hands you don't have. You celebrate folds as if you just won millions. You are entertaining above all else.

Strategy

- Play loose-aggressive: enter lots of pots, bet big regardless of hand strength
- Bluff at least 40% of the time — keep opponents guessing
- If you sense weakness (checks, small bets), attack with big raises
- Occasionally play a monster hand passively to set a trap
- Go all-in at least once every 10 hands just to keep the chaos alive

Bluffing Style

- Constantly lie about your hand in chat
- "I literally have pocket aces" (you have 7-2 offsuit)
- "I'm folding, this hand is garbage" (you're about to raise)
- React dramatically to every community card
- Taunt opponents who fold

Bet Sizing

- Pre-flop: 3-5x BB (unpredictable sizing)
- Post-flop: overbet the pot regularly (1.5-2x pot)
- When actually strong: make suspiciously small bets to look weak

`

4.2 Agent Decision Making

Each agent decision is a single LLM call with the following context:

Key rule: Agents can ONLY see their own hole cards. The spectator (user) sees all cards live, but each agent's prompt only contains its own hand. This creates the fun tension of watching an agent bluff while you can see they have nothing.

System prompt: `You are a poker agent. Your strategy file defines your personality and approach. You must respond with BOTH a chat message and an action.

Respond in this exact JSON format: { "thought": "<internal reasoning — opponents can't see this>", "chat": "<what you say out loud to the table — can be empty string>", "action": "fold" | "check" | "call" | "raise" | "all_in", "raise_amount": <number, only if action is "raise"> }`

User prompt includes:

- Agent's strategy file (full contents)
- Current game state:

- Agent's hole cards - Community cards (visible ones) - Current pot size - Current bet to match - Agent's chip count - All opponents' chip counts - Betting history this round - Position (dealer, SB, BB, etc.)

- Chat history (last 20 messages)
- Summary of opponent tendencies so far (e.g. "Bluffer has raised 70% of hands, Tight has folded 80% of hands")

4.3 Agent Memory (Per-Game)

Not needed for v1 since each game is a single hand. Future multi-hand mode would track:

- Opponent stats (VPIP, aggression, showdown hands)
- Chat analysis (lie detection across hands)
- Own history

---

5. Chat System

5.1 Chat Phases

Agents can chat at these moments:

- 1. **Pre-action:** Before making their betting decision (optional, 0-1 messages)
- 2. **Post-action:** After their action is announced (optional, 0-1 message)
- 3. **Showdown reaction:** After cards are revealed (1 message each)
- 4. **Between hands:** Quick reaction to previous hand result (optional)

5.2 Chat Rules

- Each chat message: max 100 characters (keeps it punchy)
- Agents CAN and SHOULD lie about their hands
- Agents can reference previous hands, call out bluffs, form grudges
- The strategy file controls how chatty/quiet they are

- `No agent can see another agent's` thought `field — only` chat

5.3 Narrator

`A non-agent narrator adds context:`

- (Narrator: Agent_Bluffer did NOT have pocket aces) `— after showdown reveals a lie`

- (Narrator: Agent_Shark has now won 5 hands in a row) `— milestones`

- (Narrator: Agent_Tight is down to 100 chips — danger zone) `— dramatic moments`

---

6. Display / Terminal UI

6.1 Layout

```
`╔═══════════════════════════════════════════════════════════════╗ ═══ 🃏 AI POKER ARENA
═══ ═══ Pot: $340 | Board: [K♠ 7♥ 2♦ Q♣ ___] ═══
═══╠═══════════════════════════════════════════════════════════════╣═══ ═══ ═══ 🦈 Shark $720 [K♦
K♣] ← Dealer ═══ ═══ 🎭 Bluffer $430 [5♥ 3♠] ← Small Blind ═══ ═══ 🔒 Tight $350 [J♠ T♠] ← Big
Blind ═══ ═══ 🔥 Maniac $500 [Q♥ 9♥] ═══ ═══ ═══
═══╠═══════════════════════════════════════════════════════════════╣═══ ═══ TURN — Betting Round
═══ ═══ ┌──────────────────────────────────────────────────────────┐ ═══ ═══ │ 🎭 Bluffer: "That
queen literally hit my flush 😏" │ ═══ ═══ │ 🦈 Shark: "..." │ ═══ ═══ │ 🎭 Bluffer raises to
$200 │ ═══ ═══ │ 🔒 Tight folds │ ═══ ═══ │ 💬 Tight: "Not worth it" │ ═══ ═══ │ 🔥 Maniac: "YOLO" →
calls $200 │ ═══ ═══ │ 🦈 Shark: "You sure about that?" → raises to $400 │ ═══ ═══ │ 🎭 Bluffer:
"Oh you think you're scary?" → calls │ ═══ ═══
└──────────────────────────────────────────────────────────┘ ═══
═══╠═══════════════════════════════════════════════════════════════╣═══
```

```
╚═══════════════════════════════════════════════════════════════════╝
```

6.2 Showdown Display

```
╔═══════════════════════════════════════════════════════════════════╗
🏆 SHOWDOWN — Hand #14
Board: [K♠ 7♥ 2♦ Q♣ 9♠]

🦈 Shark: [K♦ K♣] → Three of a Kind (Kings) ← WINNER
🎭 Bluffer: [5♥ 3♠] → High Card (Queen)
🔥 Maniac: [Q♥ 9♥] → Two Pair (Queens & Nines)

(Narrator: Bluffer claimed to have a flush. Bluffer had 5-high. Bluffer is a confirmed liar.)

💬 Shark: "Told you."
💬 Bluffer: "I was SO close though"
💬 Maniac: "Rigged."

🦈 Shark wins $740
╚═══════════════════════════════════════════════════════════════════╝
```

6.3 Game Speed

- **Default:** 2-second delay between actions (watchable pace)
- **Fast mode:** 0.5-second delays
- **Step mode:** Press Enter to advance each action (for debugging/enjoying)

- `Configurable via CLI flag:` --speed fast|normal|step

---

7. Configuration

`config.py`

```
python
GAME_CONFIG = {
    "starting_chips": 1000,
    "small_blind": 10,
    "big_blind": 20,
    "max_hands": 1,  # Single hand per game (v1)
    "display_speed": "normal",  # fast | normal | step
    "show_thoughts": False,  # Debug: show agent internal reasoning
    "show_all_cards": True,  # Spectator mode: viewer sees all hole cards live
}

LLM_CONFIG = {
    "model": "claude-sonnet-4-20250514",  # Model for agent decisions
    "temperature": 0.8,  # Higher = more creative/unpredictable
    "max_tokens": 300,  # Per decision
}
```

CLI Usage

```bash
# Default game: 4 agents, 50 hands
python main.py

# Custom agents
python main.py --agents shark bluffer tight

# Fast mode with debug
python main.py --speed fast --show-thoughts

# Step-through mode
python main.py --speed step

# Custom chip count
python main.py --chips 5000 --blinds 25/50
```

---

8. LLM Integration

8.1 API Setup

- Uses the same API key/endpoint as OpenClaw (Claude via GitHub Copilot)
- Each agent decision = 1 API call
- Estimated tokens per decision: ~800 input, ~150 output
- Estimated calls per hand: ~12-16 (4 agents × 3-4 betting rounds)
- Estimated calls per game: ~12-16 (single hand)

- Very cost efficient — a game takes under a minute

8.2 Prompt Structure

Per-decision prompt (~800 tokens): `[System] You are {agent_name}. {strategy_file_contents}

[User] GAME STATE:
- Hand #14, Turn
- Your cards: K♦ K♣
- Board: K♠ 7♥ 2♦ Q♣
- Pot: $340
- Your chips: $720
- Current bet to call: $200
- Position: Dealer

OPPONENTS:
- Bluffer ($430) — raised to $200 [VPIP: 78%, Aggression: High, Caught lying 3 times]
- Tight ($350) — folded [VPIP: 22%, Aggression: Low, Reliable when betting]
- Maniac ($500) — called $200 [VPIP: 65%, Aggression: Very High, Unpredictable]

RECENT CHAT:
- Bluffer: "That queen literally hit my flush 😏"
- Tight: "Not worth it"
- Maniac: "YOLO"

Respond with JSON: {"thought": "...", "chat": "...", "action": "...", "raise_amount": ...}`

8.3 Response Parsing

- Parse JSON from LLM response
- Validate action is legal (can't check if there's a bet, can't raise less than min raise)
- If invalid action → retry once with error context
- If still invalid → force fold (agent made an illegal move)
- Extract chat message and action separately

8.4 Error Handling

- API timeout → agent auto-checks/folds
- Malformed JSON → retry with stricter prompt
- Rate limiting → add delay between calls
- Track token usage per game for cost reporting

---

9. Tech Stack

- **Python 3.11+**
- `anthropic` (or `httpx` for raw API calls) — LLM integration
- **`rich`** — Terminal UI, colored output, tables, panels
- **`pydantic`** — Data validation for game state, LLM responses
- No database — everything in memory for v1
- No external dependencies beyond those three

---

10. Development Phases

Phase 1 — Core Engine (Day 1-2)

- □ Card deck, shuffling, dealing
- □ Hand evaluation (determine winning hand)
- □ Game loop (blinds → deal → bet rounds → showdown)
- □ Pot management
- □ Unit tests for hand evaluator

Phase 2 — Agent Integration (Day 3-4)

- ☐ LLM client setup (API calls)
- ☐ Strategy file loading
- ☐ Prompt construction (game state → prompt)
- ☐ Response parsing (JSON → action)
- ☐ Action validation (legal moves only)
- ☐ Basic opponent tracking (VPIP, aggression)

Phase 3 — Chat System (Day 5)

- ☐ Chat phases (pre-action, post-action, showdown)
- ☐ Chat history management
- ☐ Narrator system (lie detection, milestones)
- ☐ Chat included in agent prompts

Phase 4 — Display & Polish (Day 6-7)

- ☐ Rich terminal UI (table display, cards, chat feed)
- ☐ Game speed controls
- ☐ CLI argument parsing
- ☐ End-of-game summary (winner, stats, best hand, biggest bluff)
- ☐ README with instructions

---

11. Future Ideas (Not in v1)

- **Web UI** — React frontend with card animations, spectator mode
- **Persistent memory** — Agents remember opponents across games
- **Tournament mode** — Multiple tables, eliminations, final table

- **Custom agent creation CLI** — python main.py --new-agent "my_agent"`
- **Agent marketplace** — Share strategy files

- **ELO rating system** — Track agent rankings over time
- **Human player mode** — Join the table yourself
- **Different LLM models per agent** — Pit GPT vs Claude vs Gemini
- **Replay system** — Save/replay games as logs
- **Discord integration** — Run games in Discord, spectate via bot
- **Blind escalation** — Tournament-style increasing blinds
- **Side pots** — Proper handling for all-in scenarios with multiple players
- **Voice narration** — TTS narrator for dramatic moments

---

12. Success Criteria

A successful v1 means:

- 1. ✅ 4 agents can complete a single hand without crashing
- 2. ✅ Each agent follows its strategy file (Shark plays tight, Bluffer bluffs, etc.)
- 3. ✅ Chat feed is entertaining — agents lie, trash talk, react
- 4. ✅ Narrator catches lies at showdown
- 5. ✅ Terminal UI is readable and fun to watch — spectator sees ALL cards
- 6. ✅ Game produces a clear winner
- 7. ✅ Editing a strategy file visibly changes agent behavior
- 8. ✅ A single game runs in under 2 minutes and costs minimal tokens

---

Spec v1.0 — March 14, 2026 *Project: AI Poker Arena* *Author: Hritwik + Claw 🪷*